



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto final

Simulación de Procesos Distribuidos usando Docker, gRPC y
Relojes de Lamport.

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

10 de Junio del 2026

Índice

1	Objetivo	1
2	Planteamiento del escenario:	1
3	Diagrama del escenario:	2
4	Documentacion de codigo:	3
4.1	Codigo proto:	3
4.1.1	Descripción del Servicio	4
4.1.2	Descripción de Mensajes	4
4.2	Codigos python:	5
4.2.1	Código 1: <code>vectorial.py</code> — Reloj Vectorial	5
4.2.2	Código 2: <code>bitacora.py</code> — Manejador de Logs	6
4.2.3	Código 3: <code>process.py</code> — Servidor gRPC	7
4.2.4	Código 4: <code>driver.py</code> — Orquestador del Escenario	13
4.2.5	Código 5: <code>cli.py</code> — Cliente Interactivo	14
4.3	Códigos docker:	18
5	Secuencia de comandos para la ejecución:	21
5.1	Preparación del entorno	21
5.2	Secuencia de comandos para el escenario esto, para la terminal 2	21
5.3	Revision de logs:	24
6	Resultados:	24
6.1	Capturas de los resultados:	24
6.2	Bitacoras:	24
7	Conclusion:	24
8	Link de video:	24
	Referencias	24



1. Objetivo

Comprender cómo se mantiene el orden lógico de eventos en sistemas distribuidos mediante el uso de relojes vectoriales.

2. Planteamiento del escenario:

Para el escenario, se tomo base las aplicaciones de reparto de comida donde en este punto tomamos como referencia una arquitectura de microservicios, esto mas precisamente cuando un cliente hace una orden, en nuestro caso tomamos como referencia al restaurante Carls jr, donde tomamos como pedido la hamburguesa Famosa star, esto lo tomamos desde que el usuario selecciona la hamburguesa hasta que el sistema manda la notificación final de entrega, ahora bien, tomamos a los procesos como las responsabilidades.

Para esto, todo comienza en el Proceso 1 que es como tal la Aplicación Cliente, este proceso simula la UI de usuario propiamente, donde como tal el evento interno es agregar la comida al carrito de compras y el monto a pagar, despues de esto, una vez que la orden está lista, el Proceso 1 envía una solicitud de cobro mediante un mensaje punto a punto al Proceso 2 que este sera nuestro movimiento de Pago (donde se verifica si la tarjeta es verdadera y tiene fondos), transmitiendo su reloj vectorial inicial para emepzar la sincronización temporal del sistema.

Despues de esto, al recibir la solicitud, el Proceso 2 actualiza su reloj local y ejecuta un evento interno que es el de validar los datos de la tarjeta y procesar la propia transeaccion, despues tras aprobar el pago, este proceso se comunica con el Proceso 3 que en este punto lo estamos simulando como si fuera la aplicacion que se tiene en la cocina del Restaurante, enviándole la confirmación de fondos aprobados y los detalles de la misma orden, en este caso que tipo de hambrugesa va a desear el cliente, esto lo que hace es que garantiza un orden lógico, donde como tal el restaurante no comenzará a cocinar sin que el pago haya sido exitoso.

Posteriormente, el Proceso 3 registra un evento interno simulando la preparación y el empaquetado de la orden, de hecho una vez que el paquete está listo en el mostrador, la cocina enviara una notificación directa al Proceso 4 que es ya directamente la aplicación del Repartidor. Este cuarto proceso recibe la instrucción, sincroniza su vector, y lanza su propio evento interno para calcular la ruta óptima de entrega mediante el GPS hacia el domicilio del cliente. Al iniciar el viaje, el repartidor actualiza el estado de la orden enviando un mensaje al último proceso.

Ya por ultimo, tenemos al Proceso 5 que es el servidor de notificaciones, donde en este al recibir el mensaje del repartidor, este ejecuta un evento interno para registrar el Tiempo Estimado de Llegada que es el ETA, y para acabar el ciclo de la orden, el Proceso 5 emite un Broadcast hacia los cuatro procesos restantes P1, P2, P3 y P4 indicando que la orden ha sido completada y entregada de forma exitosa.



Este ultimo paso es importante, debido a que es el punto donde el sistema logra sincronizar completamente todos los estados haciendo que los cinco relojes vectoriales sean los mismos de manera simultánea.

Servicios gRPC

1. **Enviar Mensaje:** Recibe una estructura ReqEnvio con los identificadores de origen, destino y de mensaje, esto de forma estructurada como una cadena, despues de esto se incrementa la posición en el Reloj Vectorial del emisor y coloca la línea de envío en la bitácora.
2. **Recibir Mensaje:** Recibe una estructura ReqRecepcion que contiene el proceso de origen, el mensaje y el arreglo del vector, despues de esto actualiza el reloj local calculando los valores máximos, esto por de forma que lo hace componente a componente, e incrementando en 1, por ultimo registra el evento de recepción y devuelve un mensaje de confirmación con el vector del reloj con el tiempo actual.
3. **Evento Interno:** Recibe una solicitud ReqInterno con la descripción de una tarea (armado de carrito, validación de pago, preparación en cocina o cálculo de ruta GPS), despues este incrementa la posición en su reloj lógico vectorial y pone de forma síncrona la entrada en el archivo de registro.
4. **Broadcast:** Recibe una petición de difusión masiva, est emimso incrementa el reloj vectorial del emisor en 1 y atiende de manera por igual al mismo tiempo el estado actual junto con el mensaje a todos los demás procesos para sincronizar el estado de la orden.

3. Diagrama del escenario:

Descripcion del reloj vectorial:



Diagrama:

4. Documentacion de codigo:

4.1. Codigo proto:

Define cómo se comunican las diferentes partes del sistema utilizando **Protocol Buffers v3**. Para ello, se especifica el servicio **ProcesoService**, que incluye cuatro operaciones remotas (RPC) y los mensajes de entrada y salida que utiliza cada una.

```
1  syntax = "proto3";
2
3  package procesos;
4
5  service ProcesoService {
6      rpc EnviarMensaje (ReqEnvio) returns (Ack);
7      rpc RecibirMensaje (ReqRecepcion) returns (Ack);
8      rpc EventoInterno (ReqInterno) returns (Ack);
9      rpc Broadcast (ReqDifusion) returns (Ack);
10 }
11
12 message ReqEnvio {
13     string origen = 1;
14     string destino = 2;
15     string msg = 3;
16 }
17
18 message ReqRecepcion {
19     string origen = 1;
20     string msg = 2;
21     repeated int64 vector = 3;
22 }
23
24 message ReqInterno {
25     string desc = 1;
26 }
27
28 message ReqDifusion {
29     string origen = 1;
30     string msg = 2;
31 }
32
33 message Ack {
34     bool ok = 1;
35     string msg = 2;
```



```
36 repeated int64 vector = 3;  
37 }
```

Listing 1: procesos.proto - Definición IDL del servicio gRPC

4.1.1. Descripción del Servicio

RPC	Entrada	Salida	Descripción
EnviarMensaje	ReqEnvio	Ack	Solicita al nodo que envíe un mensaje a otro.
RecibirMensaje	ReqRecepcion	Ack	Notifica al nodo la llegada de un mensaje.
EventoInterno	ReqInterno	Ack	Dispara un evento local de negocio.
Broadcast	ReqDifusion	Ack	Difunde un mensaje a todos los nodos.

4.1.2. Descripción de Mensajes

Mensaje	Campo	Tag	Descripción
ReqEnvio	origen	1	ID del proceso emisor.
	destino	2	ID del proceso receptor.
	msg	3	Contenido del mensaje (JSON).
ReqRecepcion	origen	1	ID del proceso que origina el mensaje.
	msg	2	Contenido del mensaje.
	vector	3	Reloj vectorial del emisor (repeated int64).
ReqInterno	desc	1	Descripción del evento interno.
ReqDifusion	origen	1	ID del proceso que inicia la difusión.
	msg	2	Contenido a difundir.
Ack	ok	1	true si la operación tuvo éxito.
	msg	2	Mensaje descriptivo del resultado.
	vector	3	Estado actual del reloj vectorial del nodo (repeated int64).



El campo vector en ReqRecepcion y Ack utiliza `repeated int64` para enviar los valores del reloj vectorial como un arreglo. En Python, este campo se representa automáticamente como una lista de números enteros (`list[int]`).

4.2. Codigos python:

4.2.1. Código 1 : `vectorial.py` — Reloj Vectorial

Implementa la clase `RelojVectorial`, que se encarga de administrar el reloj vectorial de un nodo dentro de un sistema distribuido. Este reloj consiste en un arreglo de enteros, donde cada posición almacena el contador lógico correspondiente a uno de los procesos del sistema.

Dado un sistema con n procesos, el reloj vectorial de un proceso P_i se representa mediante un vector V de n enteros, donde cada posición almacena el contador lógico de un proceso. Su funcionamiento se basa en tres reglas principales:

- **Evento local:** el proceso incrementa su propio contador.
- **Envío de mensaje:** antes de enviar un mensaje, el proceso incrementa su contador y adjunta el reloj vectorial actual al mensaje.
- **Recepción de mensaje:** al recibir un mensaje, el proceso incrementa su contador y actualiza su reloj comparando cada posición con la recibida, conservando siempre el valor más grande.

```
1 import threading
2
3 class RelojVectorial:
4     def __init__(self, id_proc, num_proc=5):
5         self.num_proc = num_proc
6         self.idx = int(id_proc.replace("P", "")) - 1
7         self.vec = [0] * num_proc
8         self._bloqueo = threading.Lock()
9
10    def evento(self):
11        with self._bloqueo:
12            self.vec[self.idx] += 1
13            return list(self.vec)
14
15    def actualizar(self, vec_recibido):
16        with self._bloqueo:
17            self.vec[self.idx] += 1
18            for i in range(self.num_proc):
19                self.vec[i] = max(self.vec[i], vec_recibido[i])
20            return list(self.vec)
21
```



```
22     def valor(self):
23         with self._bloqueo:
24             return list(self.vec)
```

Listing 2: vectorial.py — Implementación del Reloj Vectorial

`__init__(self, id_proc, num_proc=5)`

Constructor de la clase. Inicializa el reloj vectorial del proceso, crea el vector con todos sus valores en cero y configura los elementos necesarios para acceder al reloj de forma segura desde varios hilos.

`evento(self) -> list`

Registra un evento local en el proceso. Para ello, incrementa el contador correspondiente al proceso actual y devuelve una copia del reloj vectorial actualizado. La operación está protegida mediante un Lock para evitar accesos simultáneos.

`actualizar(self, vec_recibido) -> list`

Procesa la recepción de un mensaje proveniente de otro nodo. El método incrementa el contador local y actualiza el reloj vectorial comparando cada posición con el vector recibido, conservando siempre el valor más alto. Finalmente, devuelve una copia del vector actualizado. La operación está protegida mediante un Lock.

`valor(self) -> list`

Devuelve una copia del estado actual del reloj vectorial sin realizar ninguna modificación. Este método se utiliza para consultar el reloj de forma segura y está protegido mediante un Lock.

4.2.2. Código 2: bitacora.py — Manejador de Logs

La clase Bitacora se encarga de almacenar mensajes en un archivo de registro (log) de forma segura. Para evitar problemas cuando varios hilos intentan escribir al mismo tiempo, utiliza un `threading.Lock`, garantizando que cada mensaje se guarde correctamente sin alterar el contenido del archivo.

```
1  import os
2  import threading
3
4  class Bitacora:
5      def __init__(self, ruta):
6          self.ruta = ruta
7          self._bloqueo = threading.Lock()
8          dir_ruta = os.path.dirname(self.ruta)
9          if dir_ruta:
```




```
10         os.makedirs(dir_ruta, exist_ok=True)
11
12     def agregar(self, linea):
13         linea_segura = linea.rstrip("\n")
14         with self._bloqueo:
15             with open(self.ruta, "a", encoding="utf-8") as arch:
16                 arch.write(linea_segura + "\n")
```

Listing 3: bitacora.py — Escritura concurrente de logs

__init__(self, ruta)

Constructor de la bitácora. Recibe la ubicación del archivo de registro y, si los directorios necesarios no existen, los crea automáticamente para garantizar que el archivo pueda almacenarse correctamente.

Parámetro	Tipo	Descripción
ruta	str	Ruta donde se almacenará el archivo de log.

agregar(self, linea)

Agrega una nueva línea al archivo de log de forma segura. Antes de escribir, elimina los saltos de línea sobrantes para evitar espacios en blanco innecesarios. El archivo se abre en modo **append** ({“a”}), de manera que los nuevos registros se añaden al final sin modificar el contenido existente.

4.2.3. Código 3: process.py — Servidor gRPC

Es el componente principal del sistema. Implementa el servicio gRPC ServicioProceso y se encarga de coordinar la comunicación entre los nodos. Sus funciones incluyen el envío y recepción de mensajes, la ejecución de eventos internos, la difusión de mensajes a todos los nodos (*broadcast*) y la actualización del reloj vectorial cada vez que ocurre alguna de estas operaciones.

Funciones Auxiliares

```
1  def leer_nodos(crudos):
2      nodos = {}
3      if not crudos: return nodos
4      items = [i.strip() for i in crudos.split(",") if i.strip()]
5      for i in items:
6          if "=" not in i: continue
7          id_nodo, dir_nodo = i.split("=", 1)
8          nodos[id_nodo.strip()] = dir_nodo.strip()
9      return nodos
10
11 def obt_env(clave, defecto=None):
```



```
12     val = os.getenv(clave)
13     return val if val not in (None, "") else defecto
14
15 def ruta_log(id_proc):
16     return obt_env("LOG_PATH", f"/app/logs/{id_proc}.log")
```

Listing 4: process.py — Funciones auxiliares

Función	Descripción
leer_nodos(crudos)	Convierte una cadena con el formato ID=host:puerto en un diccionario que relaciona cada proceso con el host y puerto que utiliza para comunicarse.
obt_env(clave, defecto)	Obtiene el valor de una variable de entorno. Si la variable no existe o está vacía, devuelve el valor indicado por defecto.
ruta_log(id_proc)	Genera la ruta del archivo de log asociado al proceso. Si la variable de entorno LOG_PATH está definida, se utiliza como directorio base para almacenar el archivo.

Clase ServicioProceso

```
1 COLORES = {
2     "P1": "\033[94m", "P2": "\033[92m", "P3": "\033[93m",
3     "P4": "\033[95m", "P5": "\033[96m",
4 }
5 FIN_COL = "\033[0m"
6
7 class ServicioProceso(procesos_pb2_grpc.ProcesoServiceServicer):
8     def __init__(self, id_proc, nodos, bita):
9         self.id_proc = id_proc
10        self.nodos = nodos
11        self.bita = bita
12        self.reloj = RelojVectorial(id_proc, len(nodos))
13        self.color = COLORES.get(id_proc, "")
14
15    def _log(self, linea):
16        self.bita.agregar(linea)
17        print(f"{self.color}{linea}{FIN_COL}", flush=True)
18
19    def _tarea(self, desc):
20        txt = desc.lower()
21        if "carrito" in txt or "armar" in txt:
22            return '{"accion":"crear_carrito", "item":"Super Star", "precio":250}'
23        if "pago" in txt or "cobro" in txt or "tarjeta" in txt:
24            return '{"accion":"validar_tarjeta", "status":"ok", "auth":"TXN-9981"}'
```



```
25     if "cocina" in txt or "preparar" in txt:
26         return '{"accion":"cocina", "status":"empaquetado", "temp":"caliente"}'
27     if "ruta" in txt or "gps" in txt:
28         return '{"accion":"gps", "destino":"Tlalnepantla", "dist":"4.2km"}'
29     if "tiempo" in txt or "notificacion" in txt:
30         return '{"accion":"push", "disp":"Celular", "status":"enviado"}'
31     return f'{{"len_desc":{len(desc)}}}'
```

Listing 5: process.py — Constructor y métodos privados

Constructor `__init__`

Inicializa el servicio del proceso, configurando su identificador, la información de los demás nodos del sistema, la bitácora y las estructuras necesarias para gestionar la comunicación y el reloj vectorial.

Parámetro	Tipo	Descripción
id_proc	str	Identificador del proceso (por ejemplo, "P1").
nodos	dict	Diccionario con la información de conexión de todos los procesos.
bita	Bitacora	Instancia utilizada para registrar eventos en el archivo de log.

Método privado `_log(linea)`

Registra una línea de texto tanto en la bitácora como en la consola. Además, utiliza colores para diferenciar los mensajes de cada proceso y fuerza la impresión inmediata mediante `flush=True`.

Método privado `_tarea(desc)`

Simula el trabajo realizado por el nodo a partir de la descripción de un evento. Dependiendo del contenido de la descripción, ejecuta una acción específica y devuelve el resultado en formato JSON.

Métodos gRPC del Servicer

```
1     def _enviar(self, destino, msg):
2         if destino not in self.nodos:
3             return False, f"Desconocido: {destino}", self.reloj.valor()
4         if destino == self.id_proc:
5             return False, "Auto-envio", self.reloj.valor()
6
7         vec = self.reloj.evento()
8         str_v = str(vec).replace(" ", "")
```

```
9      self._log(f'[SEND] {self.id_proc} -> {destino} msg="{msg}" vector  
10          r={str_v}')  
11  
12      dir_nodo = self.nodos[destino]  
13      req = procesos_pb2.ReqRecepcion(origen=self.id_proc, msg=msg, vector=vec)  
14      try:  
15          with grpc.insecure_channel(dir_nodo) as canal:  
16              stub = procesos_pb2_grpc.ProcesoServiceStub(canal)  
17              res = stub.RecibirMensaje(req)  
18              str_ack = str(list(res.vector)).replace(" ", "")  
19              self._log(f'[ACK] {self.id_proc} <- {destino} '  
20                  f'confirmacion="{res.msg}" vector_ack={str_ack}')21              return True, res.msg, list(res.vector)  
22      except grpc.RpcError as err:  
23          return False, err.details() or "Error gRPC", self.reloj.valor()  
24  
25      def EnviarMensaje(self, req, ctx):  
26          if req.origen and req.origen != self.id_proc:  
27              ctx.set_code(grpc.StatusCode.INVALID_ARGUMENT)  
28              return procesos_pb2.Ack(ok=False, msg="origen invalido",  
29                  vector=self.reloj.valor())  
30          ok, msg, _ = self._enviar(req.destino, req.msg)  
31          if not ok: ctx.set_code(grpc.StatusCode.FAILED_PRECONDITION)  
32          return procesos_pb2.Ack(ok=ok, msg=msg, vector=self.reloj.valor())  
33  
34      def RecibirMensaje(self, req, ctx):  
35          vec_rec = list(req.vector)  
36          str_rec = str(vec_rec).replace(" ", "")  
37          vec_act = self.reloj.actualizar(vec_rec)  
38          str_act = str(vec_act).replace(" ", "")  
39          self._log(f'[RECEIVE] {self.id_proc} <- {req.origen} msg="{req.msg}" '  
40              f'vector_recibido={str_rec} vector_actualizado={str_act}')41          return procesos_pb2.Ack(ok=True, msg="ACK", vector=vec_act)
```

Listing 6: process.py — Envío y recepción de mensajes



Método	Descripción
<code>_enviar(destino, msg)</code>	Método interno encargado de enviar un mensaje a otro nodo mediante gRPC. Antes del envío actualiza el reloj vectorial, realiza la comunicación y registra la respuesta recibida. También maneja posibles errores durante la comunicación.
<code>EnviarMensaje(req, ctx)</code>	Método gRPC que recibe una solicitud de envío de mensaje. Verifica la información recibida, utiliza el mecanismo de envío correspondiente y devuelve una confirmación de la operación.
<code>RecibirMensaje(req, ctx)</code>	Método gRPC ejecutado cuando un nodo recibe un mensaje. Actualiza el reloj vectorial con la información recibida, registra el evento y devuelve una confirmación junto con el estado actualizado del reloj.

Evento interno y Broadcast

```
1  def EventoInterno(self, req, ctx):
2      desc = req.desc or "evento"
3      res = self._tarea(desc)
4      vec = self.reloj.evento()
5      str_v = str(vec).replace(" ", "")
6      self._log(f'[INTERNAL] {self.id_proc} vector={str_v}')
7      return procesos_pb2.Ack(ok=True, msg=f"Registro: {res}", vector=vec)
8
9  def Broadcast(self, req, ctx):
10     if req.origen and req.origen != self.id_proc:
11         ctx.set_code(grpc.StatusCode.INVALID_ARGUMENT)
12         return procesos_pb2.Ack(ok=False, msg="origen invalido",
13                                 vector=self.reloj.valor())
14
15     msgs = []
16     vec = self.reloj.evento()
17     str_v = str(vec).replace(" ", "")
18
19     for dest in sorted(self.nodos.keys()):
20         if dest == self.id_proc: continue
21         self._log(f'[SEND] {self.id_proc} -> {dest} msg="{req.msg}" '
22                 f'vector r={str_v}')
23         dir_nodo = self.nodos[dest]
24         peticion = procesos_pb2.RegRecepcion(origen=self.id_proc,
25                                             msg=req.msg, vector=vec)
26
27         try:
28             with grpc.insecure_channel(dir_nodo) as canal:
29                 stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
```



```
28         stub.RecibirMensaje(peticion)
29         msgs.append(f"{dest}:OK")
30     except Exception:
31         msgs.append(f"{dest}:ERR")
32
33     resumen = ", ".join(msgs) if msgs else "Sin destinos"
34     return procesos_pb2.Ack(ok=True, msg=resumen, vector=self.reloj.valor())
```

Listing 7: process.py — Evento interno y Broadcast

Método	Descripción
EventoInterno(req, ctx)	Simula una actividad realizada únicamente por el nodo local. Ejecuta la tarea asociada a la descripción recibida, actualiza el reloj vectorial y registra el evento en la bitácora. Devuelve el resultado de la tarea en la respuesta.
Broadcast(req, ctx)	Envía el mismo mensaje a todos los demás nodos del sistema. Antes de iniciar los envíos, obtiene un único valor del reloj vectorial para que todos los destinatarios reciban el mensaje con la misma marca de tiempo lógica.

Función de Arranque iniciar()

```
1  def iniciar():
2      id_proc = obt_env("PROCESS_ID")
3      if not id_proc: raise SystemExit("Falta PROCESS_ID")
4
5      nodos = leer_nodos(obt_env("PEERS", ""))
6      if not nodos: raise SystemExit("Falta PEERS")
7
8      pto = int(obt_env("PORT", puerto))
9      bita = Bitacora(ruta_log(id_proc))
10
11     servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
12     procesos_pb2_grpc.add_ProcesoServiceServicer_to_server(
13         ServicioProceso(id_proc, nodos, bita), servidor)
14     servidor.add_insecure_port(f"0.0.0.0:{pto}")
15     servidor.start()
16
17     col = COLORES.get(id_proc, "")
18     print(f"{col}Nodo {id_proc} activo en el puerto {pto}...{FIN_COL}", flush=True)
19
20     try:
21         while True: time.sleep(3600)
```



```
22 except KeyboardInterrupt:
23     servidor.stop(0)
```

Listing 8: process.py — Arranque del servidor gRPC

Lee las variables de entorno `PROCESS_ID`, `PEERS` y `PORT` para obtener la configuración del nodo. Después, crea y pone en marcha el servidor gRPC utilizando un `ThreadPoolExecutor` con 10 hilos de trabajo. El proceso permanece en ejecución mediante un bucle con pausas periódicas (`sleep`) y, si se recibe una señal de interrupción (`KeyboardInterrupt`), el servidor se detiene de forma ordenada antes de finalizar.

4.2.4. Código 4: driver.py — Orquestador del Escenario

Script independiente que ejecuta automáticamente la simulación completa del proceso de entrega de comida. Para ello, realiza las llamadas a los RPCs de los distintos nodos en el orden establecido, incorporando pausas de un segundo entre cada evento para reproducir la secuencia de interacciones del sistema.

```
1 import argparse, os, time, grpc
2 import procesos_pb2, procesos_pb2_grpc
3
4 def leer_nodos(crudos):
5     # (misma logica que en process.py)
6     nodos = {}
7     if not crudos: return nodos
8     items = [i.strip() for i in crudos.split(",") if i.strip()]
9     for i in items:
10         if "=" not in i: continue
11         id_nodo, dir_nodo = i.split("=", 1)
12         nodos[id_nodo.strip()] = dir_nodo.strip()
13     return nodos
14
15 def evt_interno(nodos, dest, desc):
16     dir_nodo = nodos[dest]
17     with grpc.insecure_channel(dir_nodo) as canal:
18         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
19         stub.EventoInterno(procesos_pb2.ReqInterno(desc=desc))
20
21 def evt_enviar(nodos, origen, dest, msg):
22     dir_nodo = nodos[origen]
23     with grpc.insecure_channel(dir_nodo) as canal:
24         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
25         stub.EnviaMensaje(procesos_pb2.ReqEnvio(
26             origen=origen, destino=dest, msg=msg))
27
28 def evt_difusion(nodos, origen, msg):
29     dir_nodo = nodos[origen]
```



```
30     with grpc.insecure_channel(dir_nodo) as canal:
31         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
32         stub.Broadcast(procesos_pb2.ReqDifusion(origen=origen, msg=msg))
33
34     def escenario(nodos):
35         print("Iniciando Escenario de Entrega...", flush=True)
36         evt_interno(nodos, "P1", "armar carrito pedido");           time.sleep(1)
37         evt_enviar(nodos, "P1", "P2", '{"id":"ORD-77X","total":250,"req":"cobro"}')
38         time.sleep(1)
39         evt_interno(nodos, "P2", "validar cobro tarjeta");           time.sleep(1)
40         evt_enviar(nodos, "P2", "P3", '{"id":"ORD-77X","pago":"ok","item":"Super
41             Star"}')
42         time.sleep(1)
43         evt_interno(nodos, "P3", "preparar comida");                 time.sleep(1)
44         evt_enviar(nodos, "P3", "P4", '{"id":"ORD-77X","paquete":"listo"}')
45         time.sleep(1)
46         evt_interno(nodos, "P4", "calcular ruta gps");               time.sleep(1)
47         evt_enviar(nodos, "P4", "P5", '{"id":"ORD-77X","repartidor":"va","eta":"15m"}')
48         time.sleep(1)
49         evt_interno(nodos, "P5", "enviar notificacion");             time.sleep(1)
50         evt_difusion(nodos, "P5", '{"id":"ORD-77X","fin":"entregado","eta":"15m"}')
51         time.sleep(1)
```

Listing 9: driver.py — Orquestador del escenario de entrega

Función	Descripción
evt_interno(nodos, dest, desc)	Ejecuta un evento interno en el nodo indicado por dest, utilizando la descripción proporcionada.
evt_enviar(nodos, origen, dest, msg)	Solicita al nodo origen el envío de un mensaje hacia el nodo dest.
evt_difusion(nodos, origen, msg)	Solicita al nodo origen que envíe el mismo mensaje a todos los demás nodos del sistema.
escenario(nodos)	Ejecuta de forma automática la secuencia completa de eventos del escenario de entrega, siguiendo el orden establecido e incorporando una pausa de un segundo entre acciones para permitir el procesamiento de los nodos.

4.2.5. Código 5: cli.py — Cliente Interactivo

Herramienta de línea de comandos que permite interactuar de forma manual con cualquier nodo del sistema. Su uso evita la necesidad de modificar el código para realizar pruebas o ejecutar operaciones específicas, ya



que emplea argparse para gestionar distintos subcomandos y opciones de ejecución.

```
1 import argparse, os, grpc
2 import procesos_pb2, procesos_pb2_grpc
3
4 def leer_nodos(crudos):
5     nodos = {}
6     if not crudos: return nodos
7     items = [i.strip() for i in crudos.split(",") if i.strip()]
8     for i in items:
9         if "=" not in i: continue
10        id_nodo, dir_nodo = i.split("=", 1)
11        nodos[id_nodo.strip()] = dir_nodo.strip()
12    return nodos
13
14 def res_destino(dest, nodos):
15     if not dest: return None
16     if dest in nodos: return nodos[dest]
17     if ":" in dest: return dest
18     return None
19
20 def obt_origen(args, nodos):
21     if args.origen: return args.origen
22     if args.destino in nodos: return args.destino
23     return os.getenv("PROCESS_ID", "")
24
25 def ejecutar():
26     p = argparse.ArgumentParser(description="Cliente CLI")
27     p.add_argument("--destino", required=True)
28     p.add_argument("--nodos", default=os.getenv("PEERS", ""))
29     p.add_argument("--origen", default="")
30
31     subs = p.add_subparsers(dest="cmd", required=True)
32     cmd_int = subs.add_parser("interno")
33     cmd_int.add_argument("--desc", default="evento")
34
35     cmd_env = subs.add_parser("enviar")
36     cmd_env.add_argument("--receptor", required=True)
37     cmd_env.add_argument("--msg", required=True)
38
39     cmd_dif = subs.add_parser("difusion")
40     cmd_dif.add_argument("--msg", required=True)
41
42     args = p.parse_args()
43     nodos = leer_nodos(args.nodos)
44     dir_dest= res_destino(args.destino, nodos)
```

```
45     if not dir_dest: raise SystemExit("Destino invalido")
46
47     id_ori = obt_origen(args, nodos)
48     if args.cmd in {"enviar", "difusion"} and not id_ori:
49         raise SystemExit("Falta origen")
50
51     with grpc.insecure_channel(dir_dest) as canal:
52         stub = procesos_pb2_grpc.ProcesoServiceStub(canal)
53         if args.cmd == "interno":
54             res = stub.EventoInterno(procesos_pb2.ReqInterno(desc=args.desc))
55         elif args.cmd == "enviar":
56             res = stub.EnviarMensaje(procesos_pb2.ReqEnvio(
57                 origen=id_ori, destino=args.receptor, msg=args.msg))
58         else:
59             res = stub.Broadcast(procesos_pb2.ReqDifusion(
60                 origen=id_ori, msg=args.msg))
61
62     st = "OK" if res.ok else "ERR"
63     str_v = str(list(res.vector)).replace(" ", "")
64     print(f"{st}: {res.msg} (vector={str_v})")
```

Listing 10: cli.py — Cliente de línea de comandos

Para mantener una estructura modular y resolver las conexiones de forma dinámica, el cliente divide su lógica en las siguientes funciones:

- **leer_nodos:** Procesa una cadena de texto en crudo (típicamente obtenida de la variable de entorno PEERS) y la transforma en un diccionario que asocia el identificador de cada nodo con su dirección física.
- **res_destino:** Resuelve la dirección a la cual se conectará el cliente. Es flexible, ya que permite buscar el destino en el diccionario de nodos conocidos o aceptar directamente una dirección en formato IP:puerto.
- **obt_origen:** Deduce qué identificador debe utilizarse como remitente del mensaje. Si no se indica explícitamente en los argumentos, intenta inferirlo basándose en el destino o, como último recurso, lee la variable de entorno PROCESS_ID.
- **ejecutar:** Orquesta el flujo principal de la herramienta. Valida que los parámetros obligatorios existan y establece un canal de comunicación (`grpc.insecure_channel`) con el destino. Una vez que el nodo remoto procesa la solicitud (evento interno, envío o difusión), esta función captura la respuesta y muestra en consola el estado de la operación junto con el estado del reloj vectorial resultante.



Subcomando	Flags	Descripción
interno	-desc	Ejecuta un evento interno en el nodo seleccionado utilizando la descripción proporcionada.
enviar	-receptor, -msg	Solicita al nodo seleccionado que envíe un mensaje al receptor especificado.
difusion	-msg	Solicita al nodo seleccionado que envíe el mismo mensaje a todos los demás nodos del sistema.

Ejemplo de uso

```
python cli.py --destino P1 --nodos "P1=localhost:50051,P2=localhost:50052" \  
  enviar --receptor P2 --msg '{"prueba": true}'
```



4.3. Códigos docker:

Dockerfile:

El archivo Dockerfile lo que hace es que define las instrucciones de como se ira contruyendo la imagen base de los contenedores.

Para esto la primer alinea, lo que ahce es que inicia una imagen optimizada de Python python:3.11-slim, depsues de esto, establece /app como el directorio de trabajo principal dentro del entorno virtual.

Despues de esto, transfiere el archivo de requerimientos e instala las dependencias mediante pip install, utilizando la bandera --no-cache-dir, esto se hace mas que nada para evitar almacenar archivos temporales y reduce el peso del contenedor final, despues de esto, copia los directorios de código que en neustro los tenemos por carpetas separadas, que son proto y src, y configura la variable de entorno PYTHONPATH esto se hace para que el intérprete reconozca los módulos importados.

No obstante en la 13 linea, tenemos lo mas importante, que es directaemtne el lanzamiento de el servidor gRPC, esto por medio de la compilacion inicial del archivo proto, este lo que hara es que procesa el archivo procesos.proto para generar las clases cliente y servidor en Python.

Finalmente, declaramos el uso del puerto 50051 para permitir la comunicacion de el gRPC y definemos como comando por defecto la ejecución del servidor principal src/process.py.

Listing 11: Código dockerfile

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt ./
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY proto ./proto
9 COPY src ./src
10
11 ENV PYTHONPATH=/app/src
12
13 RUN python -m grpc_tools.protoc -I ./proto \
14     --python_out=./src \
15     --grpc_python_out=./src \
16     ./proto/procesos.proto
17
18 EXPOSE 50051
19
20 CMD ["python", "src/process.py"]
```



Docker Compose:

El archivo docker-compose es el que se encarga de organizar directamente toda la estructura completa del sistema, definiendo desde la red aislada y un total de seis servicios, donde en este caso son cinco procesos de procesamiento p1 a p5 y un proceso o nodo que es el cliente.

Para cada uno de los procesos servidores, el código lo que hace, es que indica que la imagen debe construirse a partir del directorio actual y le asigna un nombre de contenedor, esto por medio del comando environment, después de esto se colocan ya directamente las variables de entorno para la lógica del código en Python, que en este caso son las de los requisitos del proyecto, tales como el identificador único del proceso PROCESS_ID, el mapa de la red indicando las rutas de todos los nodos y la ruta de destino para almacenar directamente las bitácoras.

Para asegurar que los archivos generados no se pierdan al apagar los contenedores, se utiliza columnetes directamente, que lo que hace es que, mapea un directorio local hacia dentro del propio contenedor.

No obstante, el servicio cliente es casi igual, pero sobrescribe el comando de inicio por defecto con "sleep." "infinity", esto es por que lo que queremos es que mantenga en un estado de reposo para poder seguir mintiendo comandos de prueba, que son los **docker exec**.

Finalmente, en la parte inferior del archivo, se define la red_privada bajo un formato de tipo puente con una subred que en este caso es parecido al usado directamente en clase en los ejercicios, siendo para este proyecto el 172.21.0.0/16, lo cual permite usar direcciones estáticas para cada contenedor.

Listing 12: Código docker compose

```
1 services:
2   p1:
3     build: .
4     container_name: p1
5     environment:
6       PROCESS_ID: P1
7       PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
8       LOG_PATH: /app/logs/p1.log
9     volumes:
10      - ./logs:/app/logs
11     networks:
12       red_privada:
13         ipv4_address: 172.21.0.2
14
15   p2:
16     build: .
17     container_name: p2
18     environment:
19       PROCESS_ID: P2
20       PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
21       LOG_PATH: /app/logs/p2.log
```



```
22 volumes:
23   - ./logs:/app/logs
24 networks:
25   red_privada:
26     ipv4_address: 172.21.0.3
27
28 p3:
29   build: .
30   container_name: p3
31   environment:
32     PROCESS_ID: P3
33     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
34     LOG_PATH: /app/logs/p3.log
35   volumes:
36     - ./logs:/app/logs
37   networks:
38     red_privada:
39       ipv4_address: 172.21.0.4
40
41 p4:
42   build: .
43   container_name: p4
44   environment:
45     PROCESS_ID: P4
46     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
47     LOG_PATH: /app/logs/p4.log
48   volumes:
49     - ./logs:/app/logs
50   networks:
51     red_privada:
52       ipv4_address: 172.21.0.5
53
54 p5:
55   build: .
56   container_name: p5
57   environment:
58     PROCESS_ID: P5
59     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
60     LOG_PATH: /app/logs/p5.log
61   volumes:
62     - ./logs:/app/logs
63   networks:
64     red_privada:
65       ipv4_address: 172.21.0.6
66
67 cliente:
68   build: .
69   container_name: cliente
70   command: ["sleep", "infinity"]
71   environment:
72     PEERS: P1=p1:50051,P2=p2:50051,P3=p3:50051,P4=p4:50051,P5=p5:50051
73   volumes:
```



```
74 - ./logs:/app/logs
75 networks:
76   red_privada:
77     ipv4_address: 172.21.0.10
78
79 networks:
80   red_privada:
81     driver: bridge
82     ipam:
83       config:
84         - subnet: 172.21.0.0/16
```

5. Secuencia de comandos para la ejecución:

5.1. Preparación del entorno

Levantar los contenedores: Se levanta todo el sistema la mediante, el siguiente comando, donde la bandera -d libera la terminal.

```
docker compose up -d -build
```

Ver los logs en vivo (Terminal 1):

En dado caso, de que si quiera ver los logs, en vivo se hara uso de dos terminal, no obstante como bien sabemos los logs se guardan en una carpeta llamada logs, y se puede ver con el uso del comando cat, regresando a la terminal 1, se dejara con este comando activo. Esto permite observar cómo los contenedores reaccionan en tiempo real.

```
docker compose logs -f
```

5.2. Secuencia de comandos para el escenario esto, para la terminal 2

1. P1 (App Cliente) – Evento interno: crear el carrito

Inicia el evento interno en P1 simulando que la app del cliente crea la orden.

P1 incrementa su propia posición en el reloj vectorial, pasando de $[0,0,0,0,0]$ a $[1,0,0,0,0]$.

en este punto no hay comunicación con otros procesos.

Comando:

```
docker exec -it cliente python src/cli.py --destino P1 interno --desc "armar carrito pedido"
```



2. P1 → P2 - solicitud de cobro

P1 envía un mensaje gRPC punto a punto a P2 con los datos de la orden.

Al enviarlo, P1 incrementa su reloj nuevamente y envía su vector al mensaje.

Al recibirlo, P2 compara los vectores numero a numero del arreglo, toma los máximos e incrementa su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P1 enviar --receptor P2 --msg '{"id":"ORD-77X",  
"total":250, "req":"cobro"}
```

3. P2 (Verficiacion de Pago)

P2 ejecuta un evento interno simulando que se verifica si tarjeta de crédito tiene fondos y es real.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 interno --desc "validar cobro tarjeta"
```

4. P2 → P3 - Confirmacion al restaurante

Con el pago aprobado, P2 le envía los detalles de la orden a P3 vía gRPC.

P3 recibe el vector de P2, aplica lo mismo de revisar los máximos e incrementa su propia posición.

Su reloj ahora esta sincornizado con los eventos previos de P1 y P2.

Comando:

```
docker exec -it cliente python src/cli.py --destino P2 enviar --receptor P3 --msg '{"id":"ORD-  
77X", "pago":"ok", "item":"Super Star"}'
```

5. P3 (Restaurante)

P3 ejecuta un evento interno que lo que hace es que simula la preparación y que se empaqueta la hamburguesa.

Su reloj vectorial avanza en su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P3 interno --desc "preparar comida"
```




6. P3 → P4 - Aviso al repartidor

P3 envía un mensaje gRPC a P4 indicando que el paquete está listo para recolección.

P4 combina directamente el vector recibido con el suyo, sincronizando su reloj con todos los eventos acumulados.

Comando:

```
docker exec -it cliente python src/cli.py --destino P3 enviar --receptor P4 --msg '{"id":"ORD-77X", "paquete":"listo"}
```

7. P4 (Repartidor) - Evento interno: calcular ruta GPS

P4 realiza un evento interno que simula el cálculo de la ruta óptima hacia el destino del cliente mediante GPS.

Solo incrementa su posición en el reloj vectorial.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 interno --desc "calcular ruta gps"
```

8. P4 → P5 – Notificación de inicio de viaje

P4 le comunica a P5 que es el servidor de notificaciones que el repartidor ya está en camino.

P5 recibe el vector con el historial completo de todos los procesos y lo combinad directamente con el suyo para preparar el broadcast.

Comando:

```
docker exec -it cliente python src/cli.py --destino P4 enviar --receptor P5 --msg '{"id":"ORD-77X", "repartidor":"va", "eta":"15m"}
```

9. P5 (Servidor de Notificaciones)

P5 ejecuta un evento interno para calcular y registrar el tiempo estimado de llegada antes de mandar el broadcast.

Su reloj vectorial avanza en su propia posición.

Comando:

```
docker exec -it cliente python src/cli.py --destino P5 interno --desc "enviar notificacion"
```



10. P5 – Broadcast final a todos los nodos

P5 envía a todos al mismo tiempo el estado final de la orden a P1, P2, P3 y P4 mediante una difusión gRPC.

Cada proceso receptor compara el vector actualizado de P5 con el suyo y termina con los tiempos actualizados de forma correcta todos.

Comando:

```
docker exec -it cliente python src/cli.py --destino P5 difusion --msg '{"id":"ORD-77X", "fin":"entregado", "eta":"15m"}'
```

5.3. Revision de logs:

Para revisar el archivo de bitácora generado por cada nodo al finalizar la secuencia (se muestra el ejemplo para P1, pero aplica para P1 a P5):

```
docker exec -it p1 cat /app/logs/p1.log
```

Para detener los contenedores, esto solo se hace después de matar el proceso del comando de logs, esto baja todos los servicios de docker creados:

```
docker compose down
```

6. Resultados:

6.1. Capturas de los resultados:

6.2. Bitácoras:

7. Conclusion:

8. Link de video:

Referencias

- Docker. (2026). *Docker Documentation*. Consultado el 28 de mayo de 2026, desde <https://docs.docker.com/>
- gRPC. (2026). *gRPC Documentation*. Consultado el 28 de mayo de 2026, desde <https://grpc.io/docs/>
- Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>